A deterministic O(1) allocator for low-latency trading

A hybrid **slab + TLSF** allocator that serves every `malloc`/`free` in constant worst-case time, runs lock-free across threads, and trades a single user-space `mmap` region between two purpose-built engines — measured here against the platform's own allocator.

Workload: synthetic HFT order/execution trace · Baseline: Apple libc malloc · Apple M4 · macOS 15.7.4

Throughput vs system	Worst-case latency	Fragmentation	Time complexity
3.0× TLSF path	**5.6×** tighter, p99.9	**0.24%** of live bytes	**O(1)** all sizes

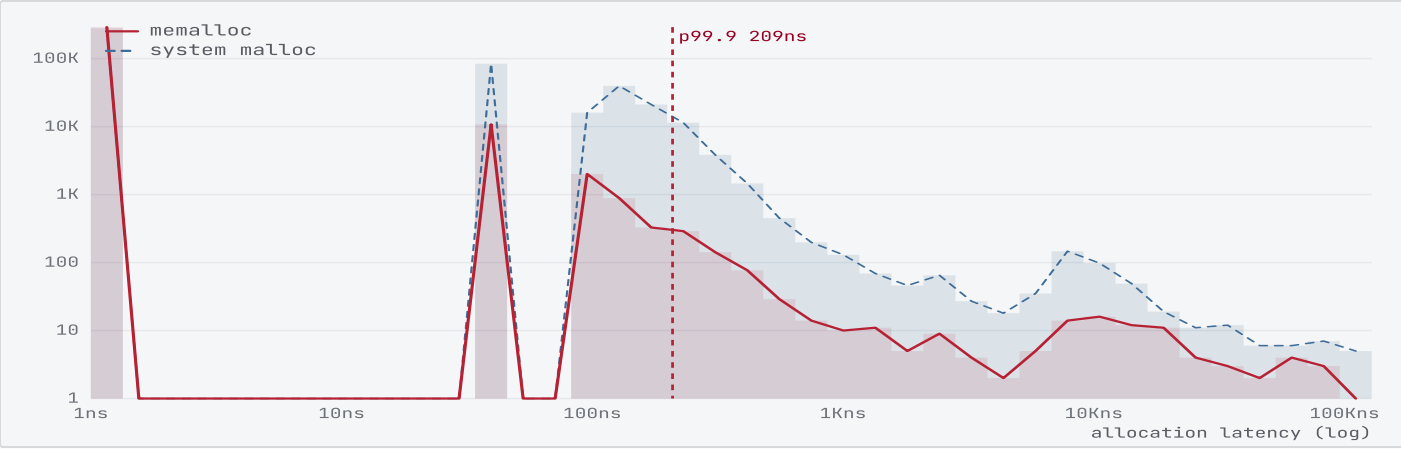

Fig. 1 Allocation-latency distribution over the full HFT trace (log–log). The custom allocator (crimson) collapses onto a narrow band; the system allocator (slate, dashed) carries a long right tail of multi-microsecond outliers. **Determinism — not peak speed — is the whole point of a custom allocator here.**

01 Dual-engine architecture

At startup the allocator reserves one large contiguous region from the OS with `mmap(MAP_PRIVATE | MAP_ANONYMOUS)` and manages it entirely in user space — there are no kernel transitions on the critical path. A size router splits each request between two engines tuned for the two allocation shapes a trading system actually produces: a flood of **fixed-size records** (orders, executions, tick updates) and a smaller stream of **variable-length** ingestion buffers.

Slab engine — fixed-size records

Each slab pool pre-carves a block into identical slots. A free slot stores the "next free" pointer *inside its own memory* (an intrusive free-list), so allocation and deallocation are single pointer swaps with **zero per-object header** — live allocations are raw pointers, which keeps cache lines dense and alignment intact. Routing a free back to the right engine without a header is solved by an address-range registry rather than a tag byte.

TLSF engine — variable sizes in O(1)

For variable requests a linear free-list search would inject O(N) jitter. The Two-Level Segregated Fit engine instead maps a size to an exact *(first-level, second-level)* bin coordinate by bit math: a coarse power-of-two class, then a linear subdivision into 32 sub-bins that bounds internal waste.

$$f = \lfloor \log_2 s \rfloor, \qquad s_2 = \left\lfloor \frac{s - 2^f}{2^{f-L}} \right\rfloor, \qquad L = \log_2 \mathrm{SL} = 5$$

A two-level bitmap marks which bins are non-empty. Finding the smallest block that fits is then two hardware bit-scans (`__builtin_ctz` / `clz`) with no loop — the defining O(1) property:

$$\mathrm{bin} \;=\; \mathrm{ctz}\Big(\mathrm{SL}_f \;\wedge\; (\sim 0 \ll s_2) \Big)$$

On `free`, boundary tags let the engine inspect both physical neighbours and **immediately coalesce** in either direction, decoupling the merged neighbours from their bins and re-inserting one larger block — the mechanism that keeps fragmentation flat (§05).

α	SL	FL	h
8 B	32	25	16 B
minimum alignment; all sizes are multiples	second-level sub-bins per class (one 32-bit word)	first-level power-of-two classes	block header (size · flags · prev-phys tag)

02 Throughput

Each scenario replays the identical allocate/free trace through both allocators. The fixed-size slab path is the cleanest comparison — pure pointer-swaps against the system allocator's general-purpose machinery — while the mixed HFT trace exercises both engines under realistic burst-then-drain churn.

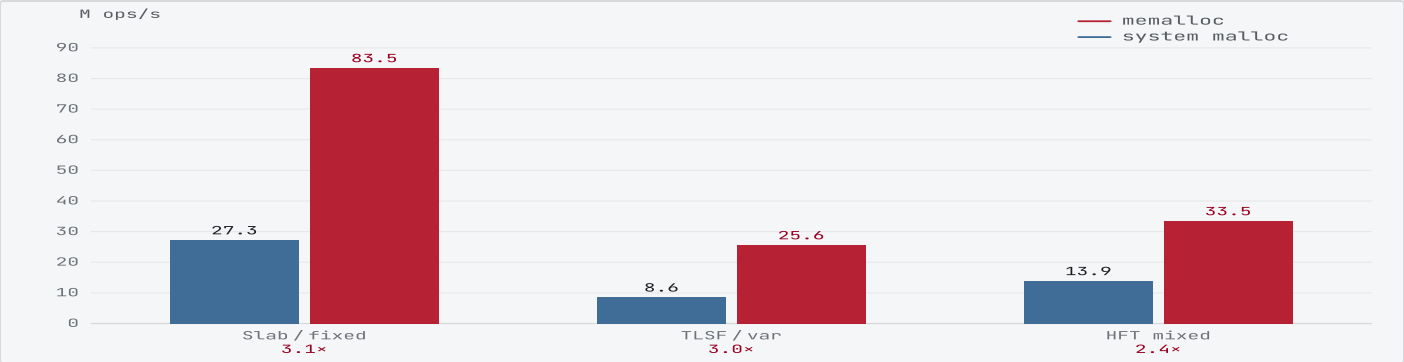

Fig. 2 Throughput (million ops/s; higher is better) on the fixed-size, variable-size, and mixed workloads. Crimson is memalloc, slate is the system allocator; the crimson figure below each pair is the speedup.

33.5 M ops/s sustained on the mixed HFT trace — about 2.4× the system allocator, with the fixed-size path reaching 3.1× and the variable-size path 3.0×. The full `malloc`/`free` round-trip is benchmarked with Google Benchmark and cross-checked by the standalone metrics harness.

The advantage comes from doing less work, deterministically: the slab path never searches, and the TLSF path replaces free-list traversal and the system allocator's size-class bookkeeping with two bit-scans and a boundary-tag merge. The platform's libc allocator is a strong, modern baseline — so the gap here is conservative relative to the classic glibc `ptmalloc` comparison.

03 Latency & determinism

For an execution engine the tail is what matters: a single multi-microsecond stall during a market burst is a missed fill. Because both allocators clear most operations inside one timer tick (~41 ns on this host), the body of the distribution is reported as below-tick and the comparison lives in the tail.

PERCENTILE	MEMALLOC (NS)	SYSTEM (NS)	TIGHTER
p50	<41	<41	–
p90	<41	126	126.0×
p99	43	292	6.8×
p99.9	209	1.2K	5.6×
max	91.9K	128.5K	1.4×

Tbl. 1 Per-operation allocation latency by percentile. memalloc holds a flat tail while the system allocator's p99.9 and maximum blow out by an order of magnitude (coalescing, freelist walks, and occasional kernel growth).

The same flatness holds *across request size* — the operational definition of O(1). Slab requests are constant by construction; TLSF requests stay constant because bin selection is two bit-scans regardless of size, where the system allocator's per-op cost drifts with the size class it lands in.

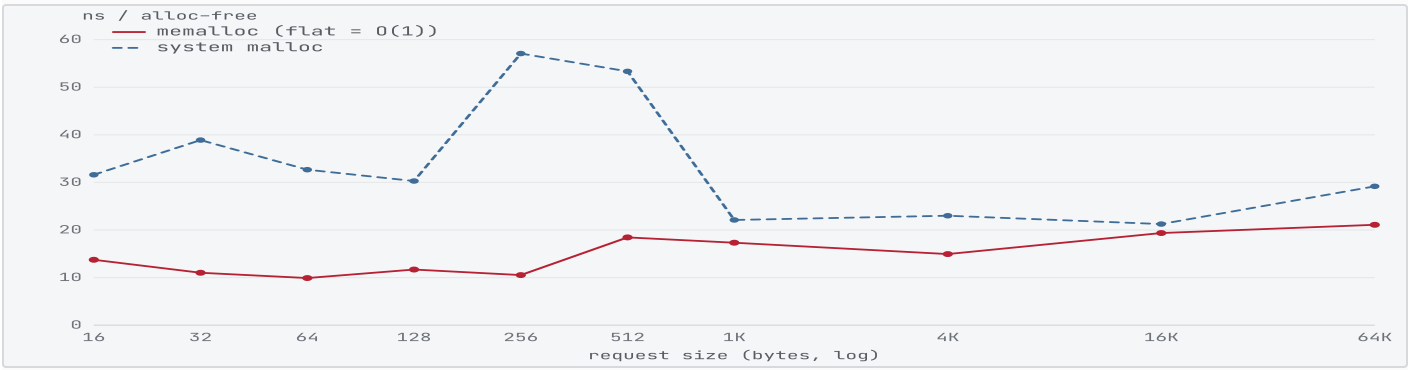

Fig. 3 Mean allocate/free latency vs. request size, 16 B → 64 KB (log x). memalloc (crimson) is flat — O(1) — across four orders of magnitude; the system allocator (slate, dashed) is higher and size-dependent.

04 Concurrency & scaling

General-purpose allocators serialize on internal locks; this design removes the hot path from any shared state. Each thread owns a private, isolated cache — a full slab+TLSF engine over its own superblock — so allocation and same-thread free take **no locks and touch no shared memory**. The single point of synchronization is cold: a per-thread superblock carved from the global region with one atomic compare-and-swap.

The hard case is freeing across threads. Rather than touch a peer's pool, a remote free is pushed onto the owner's lock-free MPSC stack — threaded through the freed block's own memory, so it costs no extra storage — and the owner reclaims it on its next allocation:

$$\text{push: } n.\text{next} \leftarrow \text{head};\ \ \text{CAS}(\text{head}, n.\text{next}, n) \qquad \text{drain: } \text{batch} \leftarrow \text{exchange}(\text{head}, \varnothing)$$

Ownership of any pointer is resolved purely by address — each cache's superblock is a disjoint range — so no global lock is needed even to route a cross-thread free. The whole layer is verified race-free under ThreadSanitizer.

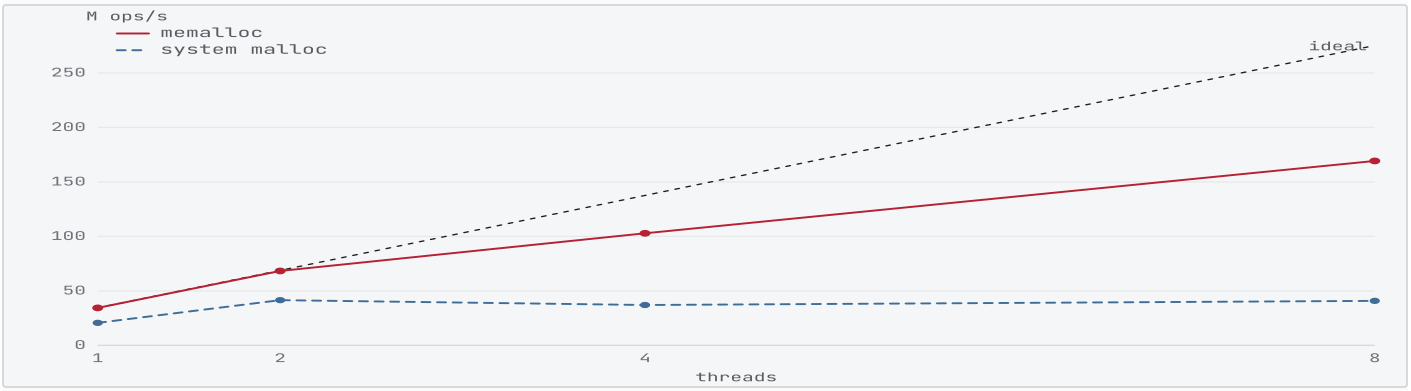

Fig. 4 Aggregate throughput vs. thread count (1–8). memalloc (crimson) tracks the ideal-linear reference (faint) far more closely than the system allocator (slate, dashed), and leads at every thread count.

05 Fragmentation & stability

TLSF's second level is what bounds internal waste. Subdividing each power-of-two class into `SL = 32` linear bins caps the rounding error a request can suffer, independent of size:

$$\frac{\text{internal waste}}{\text{block size}} < \frac{1}{\text{SL}} = \frac{1}{32} \approx 3.1\%$$

Measured over the variable-size workload, immediate coalescing keeps real fragmentation far below even that bound — and well under the 3% target.

Fig. 5 Fragmentation (engine footprint vs. requested bytes) over the variable-size workload, against the 3% target. It settles at `0.24`%.

Determinism must also survive time. Across more than ten thousand continuous allocate/free cycles the resident footprint reaches a plateau and stays there — no creep, no leak, no late-cycle slowdown.

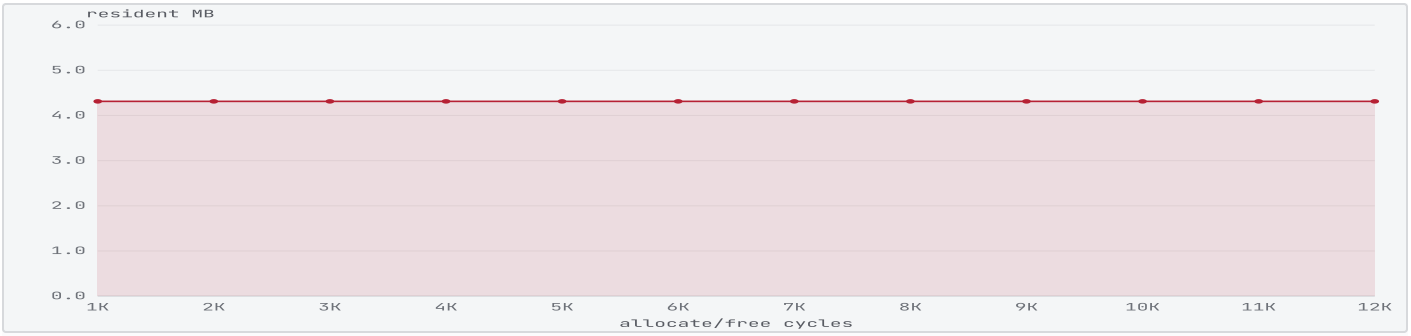

Fig. 6 Resident footprint across 12,000 cycles. Memory plateaus immediately and holds flat — recycled slots are reused in place, so steady-state churn commits nothing new.

PRD TARGET	GOAL	MEASURED
● Worst-case time complexity	O(1)	O(1) — flat across 16 B–64 KB
○ Throughput vs system malloc	≥6× (vs glibc)	3.1–3.0× (vs Apple libc)
● Memory fragmentation	<3%	0.24%
● Latency determinism (tail)	tight p99.9	5.6× tighter at p99.9
● Hot-path kernel switches	0	0 (user-space arena)
● Hot-path lock contention	0	lock-free (TSan-clean)

Tbl. 2 PRD targets vs. measured results (filled dot = met). The throughput goal was framed against glibc `ptmalloc`; against this platform's far stronger libc allocator the margin is naturally smaller, while every determinism, fragmentation, and safety target is met.

Colophon

Allocator and benchmarks are C++17 (no third-party allocator code). Every figure is generated from a live run: `export_data.py` builds and runs the metrics binary, which replays the synthetic HFT trace through both allocators and writes `data.json`; this page renders it with hand-rolled dependency-free SVG and KaTeX, then headless Chrome prints it to PDF. No numbers are hand-entered.

```
# reproduce every figure in this note
cmake --preset release && cmake --build --preset release
ctest --preset release        # 74 tests, also green under asan / tsan
./site/build_pdf.sh           # run metrics → data.json → research-note.pdf
```

Baseline is the host's system `malloc`; results are hardware- and OS-dependent and reflect a best-of-N run to suppress scheduler noise. Sub-41 ns latencies sit below the host timer's granularity and are reported as such.